

Un sistema de gestión para una tienda de cartas Magic The Gathering

Asignatura: Desarrollo Fullstack I

Integrantes: Ignacia Padilla y Elba Sánchez

Docente: Ricardo Mauricio Gonzalez Vejar

1. Introducción y Contexto.

1.1 Descripción del proyecto.

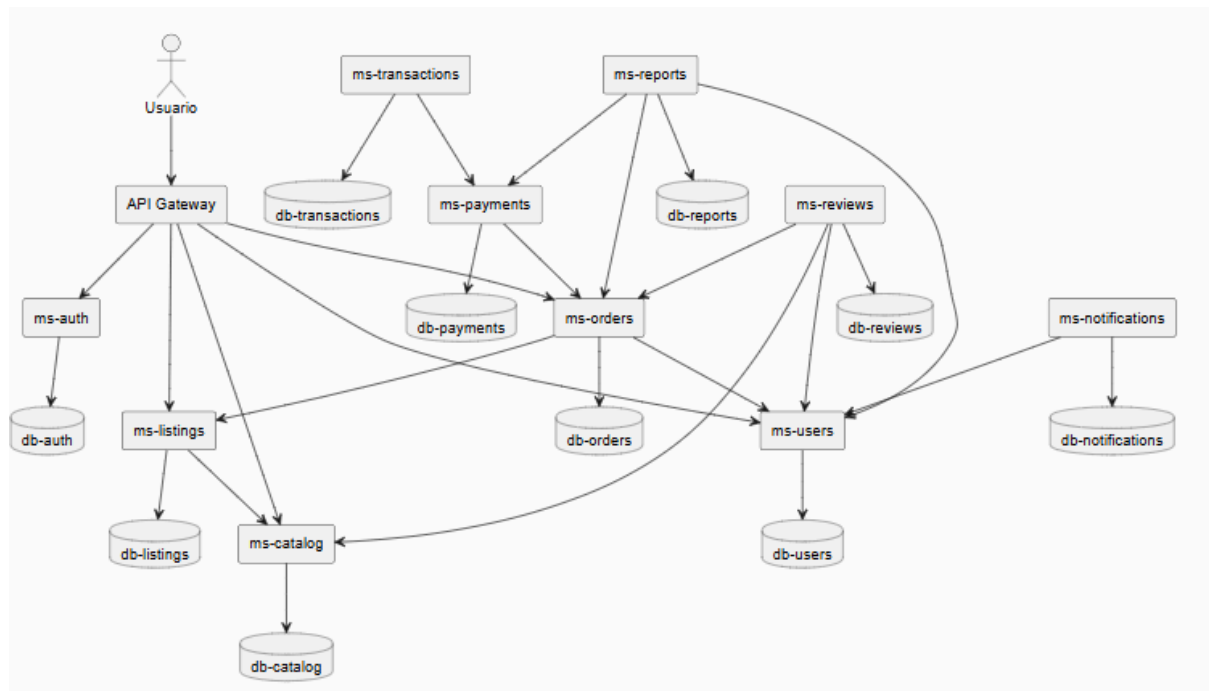
Quandrix es una solución de backend basada en una arquitectura distribuida de microservicios para un marketplace de cartas de Magic The Gathering. Permite que tiendas y usuarios individuales publiquen, compren y gestionen "singles", integrando datos externos de la API de Scryfall

2. Arquitectura del Sistema

2.1 Descripción general

El sistema fue desarrollado bajo una arquitectura de microservicios utilizando Spring Boot, permitiendo que cada módulo funcione de manera independiente.

2.2 Diagrama de Arquitectura



El sistema está compuesto por:

- API Gateway
- Eureka Server
- 10 microservicios
- Bases de datos independientes
- Comunicación mediante OpenFeign y WebClient

2.3 Roles de Usuario: Descripción de los 2 o 3 roles definidos (ej: Administrador, Cliente) y sus permisos asociados.

Administrador	Cliente/Usuario	Vendedor
Gestiona cartas (catálogo)	Compra cartas	Publica cartas (ms-listings)
Puede eliminar publicaciones	Crea pedidos	Gestiona sus publicaciones
Accede a reportes	Puede dejar reseñas (Opcional)	Recibe pagos indirectamente
Supervisa el sistema	Recibe notificaciones	Consulta sus ventas

2.4 Arquitectura de Microservicios:

Microservicio	Puerto	Función principal
Eureka-server	8761	Registro y descubrimiento de servicios.
api-gateway	8080	Punto de entrada único y enrutamiento.
ms-auth	8081	Autenticación y autorización de usuarios
ms-users	8082	Gestión de perfiles y usuarios
ms-catalog	8083	Administración del catálogo de cartas MTG
ms-listings	8084	Publicación y consulta de cartas en venta

ms-orders	8085	Gestión de órdenes de compra
ms-payments	8086	Procesamiento de pagos
ms-transactions	8087	Registro histórico de transacciones
ms-reviews	8088	Gestión de reseñas y valoraciones
ms-notifications	8089	Envío de notificaciones al usuario
ms-reports	8090	Generación de reportes y estadísticas

2.5 Diagrama Entidad-Relación (DER)

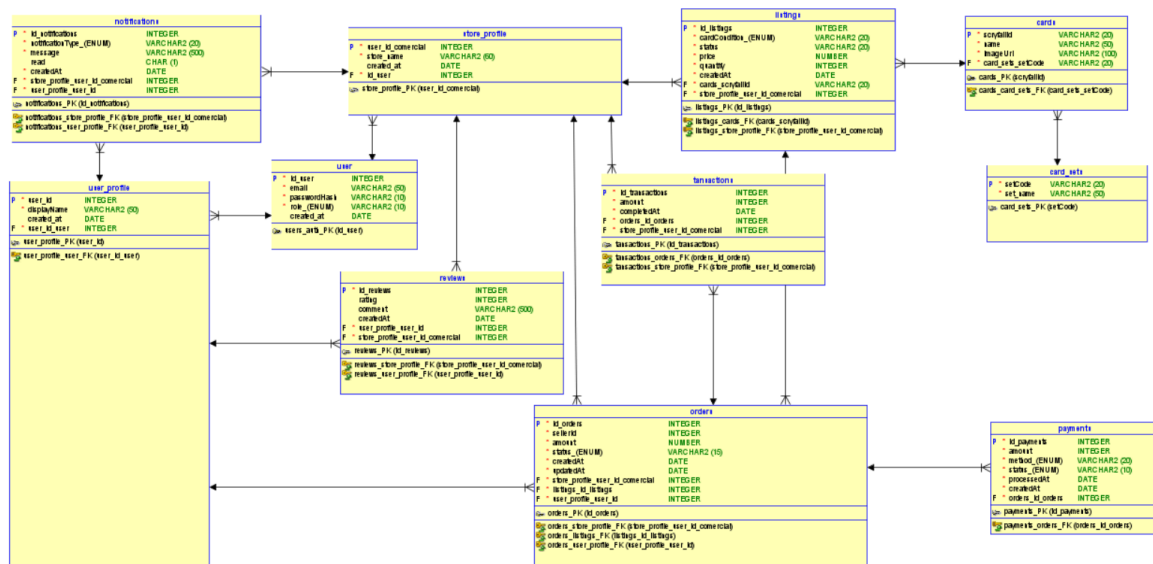


Diagrama Entidad-Relación (DER).png

3. Especificaciones Técnicas

3.1 Persistencia: Cada microservicio cuenta con su propia base de datos **MySQL**, gestionada a través de **Spring Data JPA e Hibernate**, con relaciones normalizadas y manejo de integridad referencial.

3.2 Validaciones: Implementación de **Bean Validation (JSR 380)** en DTOs y entidades para asegurar la calidad de los datos de entrada.

3.3 Comunicación: Interoperabilidad lograda mediante **OpenFeign** y **WebClient**, con manejo de timeouts y errores remotos.

Ejemplo:

1. ms-orders consulta información de usuarios mediante ms-users.
2. ms-payments valida órdenes creadas en ms-orders.
3. ms-notifications envía mensajes después de una compra exitosa.

3.4 Manejo de Errores: Uso de **@ControllerAdvice** para capturar excepciones y retornar respuestas estructuradas en formato JSON con códigos HTTP semánticos.

3.5 Logs: Trazabilidad completa mediante **SLF4J** en puntos estratégicos del flujo de negocio.

4. Tecnologías Utilizadas

4.1 Tecnologías utilizadas

- **Lenguaje:** Java 21.
- **Framework:** Spring Boot 4.0.6.
- **Gestión de Dependencias:** Maven.
- **Contenedores:** Docker & Docker Compose.
- **Seguridad:** Spring Security + JWT.

5. Flujo de Operación (Guía de Pruebas REST)

Para verificar el funcionamiento, se recomienda el siguiente flujo en Postman:

1. POST /auth/register → registrar usuario
2. POST /auth/login → obtener JWT
3. GET /catalog/search?name= → buscar carta
4. GET /catalog/{scryfallId} → obtener carta por ID
5. POST /listings → publicar listing
6. POST /orders → crear orden de compra
7. GET /payments/order/{id} → verificar pago
8. POST /reviews → dejar reseña
9. GET /reports/summary → ver resumen (ADMIN)

6. Colaboración y Entrega.

6.1 Enlace al Repositorio de GitHub: URL del repositorio donde se visualice el historial de commits y las ramas de desarrollo.

Enlace a repositorio: <https://github.com/NovaFugaz/quandrix>

El repositorio contiene:

- **Historial de commits:**
<https://github.com/NovaFugaz/quandrix/pulls?q=is%3Apr+is%3Aclosed>

- Ramas de desarrollo:

Branches

New branch

Overview

Yours

Active

Stale

All

Q

Search branches...

Default

Branch	Updated	Check status	Behind	Ahead	Pull request
main	19 minutes ago				Default

Your branches

Branch	Updated	Check status	Behind	Ahead	Pull request
dev_elba	19 hours ago		24	0	#16

Active branches

Branch	Updated	Check status	Behind	Ahead	Pull request
development	22 minutes ago		17	0	#21
dev_elba	19 hours ago		24	0	#16

- Captura del despliegue (captura docker):

[+] up 13/13

✓ Container

quandrix-mysql

Healthy

✓ Container

quandrix-eureka

Started

✓ Container

quandrix-gateway

Started

✓ Container

quandrix-transactions

Started

✓ Container

quandrix-notifications

Started

✓ Container

quandrix-auth

Started

✓ Container

quandrix-payments

Started

✓ Container

quandrix-catalog

Started

✓ Container

quandrix-users

Started

✓ Container

quandrix-reviews

Started

✓ Container

quandrix-reports

Started

✓ Container

quandrix-listings

Started

✓ Container

quandrix-orders

Started

sudo docker ps

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
e837e226b51b	quandrix-ms-orders	"java -jar app.jar"	36 seconds ago	Up 23 seconds	0.0.0.0:8085->8085/tcp, [::]:8085->8085/tcp	quandrix-orders
bc5c9d48556b	quandrix-ms-reports	"java -jar app.jar"	36 seconds ago	Up 24 seconds	0.0.0.0:8089->8089/tcp, [::]:8089->8089/tcp	quandrix-reports
75755130b401	quandrix-ms-listings	"java -jar app.jar"	36 seconds ago	Up 23 seconds	0.0.0.0:8084->8084/tcp, [::]:8084->8084/tcp	quandrix-listings
34ae68951968	quandrix-ms-reviews	"java -jar app.jar"	36 seconds ago	Up 24 seconds	0.0.0.0:8087->8087/tcp, [::]:8087->8087/tcp	quandrix-reviews
c7a30b134304	quandrix-ms-users	"java -jar app.jar"	36 seconds ago	Up 24 seconds	0.0.0.0:8082->8082/tcp, [::]:8082->8082/tcp	quandrix-users
3259e1f7dd62	quandrix-ms-payments	"java -jar app.jar"	37 seconds ago	Up 25 seconds	0.0.0.0:8086->8086/tcp, [::]:8086->8086/tcp	quandrix-payments
690e0b0e4b41	quandrix-ms-catalog	"java -jar app.jar"	37 seconds ago	Up 25 seconds	0.0.0.0:8083->8083/tcp, [::]:8083->8083/tcp	quandrix-catalog
5f5592611d80	quandrix-ms-notifications	"java -jar app.jar"	37 seconds ago	Up 25 seconds	0.0.0.0:8088->8088/tcp, [::]:8088->8088/tcp	quandrix-notifications
e5ab9f669b0a	quandrix-ms-auth	"java -jar app.jar"	37 seconds ago	Up 25 seconds	0.0.0.0:8081->8081/tcp, [::]:8081->8081/tcp	quandrix-auth
72df7029514d	quandrix-ms-transactions	"java -jar app.jar"	37 seconds ago	Up 25 seconds	0.0.0.0:8090->8090/tcp, [::]:8090->8090/tcp	quandrix-transactions
7a30d8a4bf24	quandrix-api-gateway	"java -jar app.jar"	37 seconds ago	Up 36 seconds	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp	quandrix-gateway
5003c7cd5cc7	mysql:8.0	"docker-entrypoint.s..."	37 seconds ago	Up 36 seconds (healthy)	0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp, 33060/tcp	quandrix-mysql
5bf9c10e985d	quandrix-eureka-server	"java -jar app.jar"	37 seconds ago	Up 36 seconds	0.0.0.0:8761->8761/tcp, [::]:8761->8761/tcp	quandrix-eureka

- **Captura de Eureka funcionando:**

DS Replicas

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
API-GATEWAY	n/a (1)	(1)	UP (1) - 7a30d8a4bf24:api-gateway:8080
MS-AUTH	n/a (1)	(1)	UP (1) - e5ab8f669bba:ms-auth:8081
MS-CATALOG	n/a (1)	(1)	UP (1) - 690e0b8e4b41:ms-catalog:8083
MS-LISTINGS	n/a (1)	(1)	UP (1) - 75755130b401:ms-listings:8084
MS-NOTIFICATIONS	n/a (1)	(1)	UP (1) - 5f5592611d80:ms-notifications:8088
MS-ORDERS	n/a (1)	(1)	UP (1) - e837e226b51b:ms-orders:8085
MS-PAYMENTS	n/a (1)	(1)	UP (1) - 3259e1f7dd62:ms-payments:8086
MS-REPORTS	n/a (1)	(1)	UP (1) - bc5c9d48556b:ms-reports:8089
MS-REVIEWS	n/a (1)	(1)	UP (1) - 34ae68951968:ms-reviews:8087
MS-TRANSACTIONS	n/a (1)	(1)	UP (1) - 72df70295f4d:ms-transactions:8090
MS-USERS	n/a (1)	(1)	UP (1) - c7a38bf34384:ms-users:8082

General Info

6.2 Gestión de Proyecto

Gestión de proyecto